

TASK 1 : Develop a C program using fork() to create a **parent and child process** and display:

- PID
- PPID
- Process state

```
krishnasree08@krishnasree08:~$ mkdir week3
mkdir: cannot create directory 'week3': File exists
krishnasree08@krishnasree08:~$ cd week3
krishnasree08@krishnasree08:~/week3$ pwd
/home/krishnasree08/week3
krishnasree08@krishnasree08:~/week3$ nano task3.c
krishnasree08@krishnasree08:~/week3$ gcc task3.c -o task3
krishnasree08@krishnasree08:~/week3$ ls
File1.txt File2.txt copy.c copy_program task1.c task1_program task3 task3.c
krishnasree08@krishnasree08:~/week3$ ./task3
Before fork():
PID = 634
PPID = 422

Parent Process:
PID = 634
PPID = 422
Child PID = 635
State: Running
Parent waiting for child...
Child Process:
PID = 635
PPID = 634
State: Running
Child Process: Terminated
Child has terminated.
Parent Process: Running
```

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Before fork():\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("Child Process:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State: Running\n");

        sleep(5);

        printf("Child Process: Terminated\n");
        exit(0);
    }

    else {
        // Parent process
        printf("\nParent Process:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child PID = %d\n", pid);
        printf("State: Running\n");

        printf("Parent waiting for child...\n");
        wait(NULL);

        printf("Child has terminated.\n");
        printf("Parent Process: Running\n");
    }

    return 0;
}

```

TASK 2:

Ready → Running → Waiting → Terminated

using:

- ps
- top
- /proc

```

krishnasree08@krishnasree08:~/week3$ nano task2.c
krishnasree08@krishnasree08:~/week3$ gcc task2.c -o task2
krishnasree08@krishnasree08:~/week3$ ./task2 &
[1] 786
krishnasree08@krishnasree08:~/week3$ Process started.
PID: 786
Process is running...

```

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process started.\n");
    printf("PID: %d\n", getpid());

    printf("Process is running...\n");

    sleep(20);

    printf("Process completed.\n");

    return 0;
}

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	21736	13080	9736	S	0.0	0.2	0:01.21	systemd
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.02	init-systemd(Ub
6	root	20	0	3196	2132	2008	S	0.0	0.0	0:00.01	init
55	root	19	-1	50444	16200	15104	S	0.0	0.2	0:00.32	systemd-journal
109	root	20	0	25284	6624	5124	S	0.0	0.1	0:00.56	systemd-udev
128	systemd+	20	0	21344	13288	11160	S	0.0	0.2	0:00.21	systemd-resolve
135	systemd+	20	0	91036	7992	7032	S	0.0	0.1	0:00.16	systemd-timesyn
197	root	20	0	4244	2684	2432	S	0.0	0.0	0:00.02	cron
198	message+	20	0	9540	5404	4712	S	0.0	0.1	0:00.10	dbus-daemon
217	root	20	0	17976	8776	7740	S	0.0	0.1	0:00.16	systemd-logind
251	syslog	20	0	222516	5916	4536	S	0.0	0.1	0:00.12	rsyslogd
260	root	20	0	3124	1972	1836	S	0.0	0.0	0:00.01	agetty
266	root	20	0	107044	23112	13656	S	0.0	0.3	0:00.25	unattended-upgr
420	root	20	0	3188	1104	980	S	0.0	0.0	0:00.00	SessionLeader
421	root	20	0	3204	1244	1108	S	0.0	0.0	0:00.07	Relay(422)
422	krishna+	20	0	6596	5948	3700	S	0.0	0.1	0:00.08	bash
423	root	20	0	6704	4672	3884	S	0.0	0.1	0:00.02	login
471	krishna+	20	0	20328	11516	9392	S	0.0	0.2	0:00.22	systemd
472	krishna+	20	0	21176	3576	1848	S	0.0	0.1	0:00.00	(sd-pam)
498	krishna+	20	0	6080	5368	3696	S	0.0	0.1	0:00.04	bash
921	krishna+	20	0	9296	5688	3512	R	0.0	0.1	0:00.00	top